PHASE 3 · INTERMEDIATE

DAY 15
Functional Programming

map · filter · reduce · lambda · zip · enumerate · sorted(key=)

2 HOURS	4 CONCEPTS	3 CODE FILES	19 PROBLEMS

30-DAY PYTHON PROGRAMMING COURSE
Instructor Session Guide

Session Overview

Goal: Introduce the functional programming toolkit in Python — `lambda`, `map()`, `filter()`, `reduce()`, `zip()`, `enumerate()`, and `sorted()` with a `key=` argument — so students can transform and process collections of data in a single expressive line instead of a multi-line loop.

Outcome: By the end of this session, students can write short anonymous functions with `lambda`, apply them across a Flipkart order list or a Swiggy menu with `map()`/`filter()`, combine two related lists with `zip()`, track index and value together with `enumerate()`, and sort structured records (dictionaries, tuples) by a custom rule using `sorted(key=...)`.

Session Timeline

Time	Segment	Details
0:00 – 0:10	Warm-up & Recap	Quick review of Day 13–14 (generators, decorators); introduce today's theme.
0:10 – 0:30	Lambda Expressions	Anonymous functions, syntax, when to use lambda vs def.
0:30 – 1:00	map() / filter() / reduce()	Transform, select, and accumulate over iterables functionally.
1:00 – 1:10	Break	Short breather.
1:10 – 1:35	zip() & enumerate()	Pairing iterables together and tracking index + value.
1:35 – 1:55	sorted() with key=	Custom sort rules using lambda keys, including multi-key sorts.
1:55 – 2:00	Wrap-up	Recap, assign practice set, preview Day 16.

> **TIP**
>
> Functional style is a **toolset**, not a replacement for loops. Encourage students to ask: 'Does this read more clearly as a comprehension or a loop?' Functional tools shine when the transformation is simple and the intent is 'apply this to every item.'

Prerequisites & Preparation

Requirements

Requirement	Why It's Needed
Days 1–14 completed	Functions (Day 5), data structures (Day 6), and iterators (Day 13) are used throughout today's examples.
VS Code + Python 3.12 environment	Same setup from Day 1; no new packages required — everything used today is in the standard library.
Comfortable writing regular functions with def	Lambda is introduced as a shorthand for simple functions, so students should already know def syntax.

Instructor Pre-Session Checklist

- ■ Open a fresh VS Code workspace with the Day 15 files ready to demo.
- ■ Have a Flipkart-style order list and a Swiggy-style menu list prepared as demo data.
- ■ Test that functools.reduce() import works in the demo environment (Python 3.12).
- ■ Pre-load the Day 14 recap slide (decorators/closures) for the warm-up.
- ■ Print or open the Quick Reference Card for post-session distribution.

Step-by-Step Instruction Guide

STEP 1 — Lambda Expressions

What to Say:

Sometimes you need a tiny function for one-time use — you don't want to give it a name and write three lines with def. Python's lambda lets you write a function in a single expression. Think of it as a 'throwaway' function: no name, no return keyword needed, just an expression that's automatically returned.

```
# A regular function
def square(x):
    return x * x

# The same thing as a lambda
square_lambda = lambda x: x * x
print(square_lambda(5))
# Output: 25

# Lambda with multiple arguments
calculate_price = lambda base, gst: base + (base * gst / 100)
print(calculate_price(1000, 18))  # Riya's Flipkart order
# Output: 1180.0
```

Common Errors

Error	Fix
Writing 'lambda x: return x * x' (using return inside a lambda)	Lambdas never use return — the expression itself is the return value.
Assigning a lambda to a name and treating it like a normal named function everywhere	PEP 8 discourages this — prefer def for anything that needs a name. Lambdas are best as short, throwaway arguments to map/filter/sorted.

DONE WHEN

Students can write a one-line lambda that takes one or two arguments and correctly predict its output without running it.

STEP 2 — map(), filter(), and reduce()

What to Say:

These three functions are the classic functional trio. map() applies a function to every item in an iterable. filter() keeps only the items where a function returns True. reduce() — from the functools module — combines all items

into a single running result. Let's use Arjun's Swiggy cart to see all three in action.

```
from functools import reduce

cart_prices = [249, 499, 129, 899, 349]  # Arjun's Swiggy cart, INR

# map(): apply GST (5%) to every price
with_gst = list(map(lambda p: round(p * 1.05, 2), cart_prices))
print(with_gst)
# Output: [261.45, 523.95, 135.45, 943.95, 366.45]

# filter(): keep only items over Rs 300
premium_items = list(filter(lambda p: p > 300, cart_prices))
print(premium_items)
# Output: [499, 899, 349]

# reduce(): total cart value
total = reduce(lambda acc, p: acc + p, cart_prices)
print(total)
# Output: 2125
```

Common Errors

Error	Fix
Forgetting list() around map()/filter() and printing a	map() and filter() return lazy iterators in Python 3 — wrap in list() to see the values (or iterate with a for loop).
ImportError: cannot import name 'reduce' from 'functools'	Double-check the import line — reduce moved out of builtins into functools back in Python 3.0, it's a very common one to forget.

DONE WHEN

Students can chain map() and filter() on the same list and use reduce() to collapse a list of numbers into one value (sum, product, or max).

STEP 3 — zip() and enumerate()

What to Say:

Two very common iteration helpers. zip() lets you walk through two or more lists together, pairing them up item by item — perfect for combining a list of student names with a list of their marks. enumerate() gives you the index and the value together, so you never have to write range(len(...)) again.

```
students = ["Priya", "Arjun", "Meera"]
marks = [88, 76, 92]

# zip(): pair names with marks
for name, mark in zip(students, marks):
    print(f"{name}: {mark}")
# Output: Priya: 88
#         Arjun: 76
#         Meera: 92

# enumerate(): index + value, starting at 1
for rank, name in enumerate(students, start=1):
    print(f"Rank {rank}: {name}")
# Output: Rank 1: Priya
#         Rank 2: Arjun
#         Rank 3: Meera
```

Common Errors

Error	Fix
Assuming zip() pads the shorter list with None when lengths differ	zip() silently stops at the shortest iterable — use itertools.zip_longest() (Day 13) if padding is required.
Writing for i in enumerate(students) and then trying to use i as both index and value	enumerate() yields a tuple — unpack it as for index, value in enumerate(...).

DONE WHEN

Students can zip two IRCTC-style lists (train names + platform numbers) into a dictionary using dict(zip(...)), and use enumerate() to number a printed list from 1.

STEP 4 — sorted() with key=

What to Say:

By default, sorted() compares items directly — but what if you want to sort a list of dictionaries by one field, like sorting Paytm transactions by amount? That's what the key= argument is for: it takes a function that tells sorted() what to compare.

```
transactions = [
    {"merchant": "Zomato", "amount": 450},
    {"merchant": "Ola", "amount": 180},
    {"merchant": "BookMyShow", "amount": 900},
]

# Sort by amount, ascending
by_amount = sorted(transactions, key=lambda t: t['amount'])
print([t['merchant'] for t in by_amount])
# Output: ['Ola', 'Zomato', 'BookMyShow']

# Sort by amount, descending
by_amount_desc = sorted(transactions, key=lambda t: t['amount'], reverse=True)
print([t['merchant'] for t in by_amount_desc])
# Output: ['BookMyShow', 'Zomato', 'Ola']
```

Common Errors

Error	Fix
Passing key=t['amount'] instead of key=lambda t: t['amount']	key must be a function reference, not a value — the lambda defers evaluation until sorted() calls it on each item.
Using .sort() on a tuple or expecting sorted() to modify the list in place	sorted() always returns a new list; list.sort() modifies in place but only works on lists, not tuples or other iterables.

DONE WHEN

Students can sort a list of dictionaries by a numeric field ascending and descending, and can explain in one sentence what the key function does.

Session Wrap-Up

Student Checklist

- ■ I can write a lambda expression with one or more arguments.
- ■ I can use map() to transform every item in a list.
- ■ I can use filter() to keep only items matching a condition.
- ■ I can use functools.reduce() to collapse a list into a single value.
- ■ I can pair two lists together with zip().
- ■ I can loop with index and value together using enumerate().
- ■ I can sort a list of dictionaries or tuples using sorted(key=...).

Key Concepts Recap

Concept	One-Line Summary
lambda	An anonymous, single-expression function.
map(fn, iterable)	Applies fn to every item, returns a lazy iterator.
filter(fn, iterable)	Keeps items where fn returns True.
reduce(fn, iterable)	Combines all items into one value using fn.
zip(a, b, ...)	Pairs items from multiple iterables together.
enumerate(iterable)	Yields (index, value) pairs.
sorted(iterable, key=fn)	Returns a new sorted list using fn as the comparison rule.

Next Day Preview — Day 16: Concurrency Basics

Tomorrow we shift from functional tools to doing multiple things at once: the threading and multiprocessing modules, and an introduction to async/await with asyncio.run(). Students should make sure today's functional concepts (especially lambda and map/filter) are comfortable, since async examples often combine them.

> **WATCH OUT**
>
> If students mix up map/filter/reduce, have them say it out loud: **map transforms, filter selects, reduce combines.** This one-liner resolves most of the confusion.

Quick Reference Card

Daily Workflow Commands

```
cd ~/python-course/day15
uv venv
source .venv/bin/activate      # macOS/Linux
.venv\Scripts\activate         # Windows
python day15_01_lambda_functions.py
python day15_02_map_filter_reduce.py
python day15_03_zip_enumerate_sorted.py
```

VS Code Shortcuts

Shortcut	Action
Ctrl/Cmd + `	Open integrated terminal
F5	Run current file with debugger
Shift + Enter	Run selected line(s) in terminal
Ctrl/Cmd + /	Toggle line comment

Tool Summary

Tool	Signature	Returns
lambda	lambda args: expr	Function object
map	map(fn, iterable)	Iterator
filter	filter(fn, iterable)	Iterator
reduce	reduce(fn, iterable, [init])	Single value
zip	zip(iter1, iter2, ...)	Iterator of tuples
enumerate	enumerate(iterable, start=0)	Iterator of (i, val)
sorted	sorted(iterable, key=fn, reverse=False)	New list